卒業論文　2025 年度

Doubl-Cut-And-Join 操作によるゲノム間距離に関する線形時間アルゴリズムの再実装

平田 智也
(学籍番号: 62216580)

指導教員　小田 芳彰 准教授

2026 年 1 月

慶應義塾大学理工学部数理科学科 4 年
小田研究室

目次

論文要旨 ・・・

論文要旨

生物学において進化の過程を論じる際、突然変異を経て変化する世代間の影響を考慮することは不可欠である。また、生物学的背景から発生しやすい変異とそうでない変異を区別し、前者を中心に遺伝的な距離を定義することは自然なアプローチといえる。

Hannenhalli および Pevzner (1995)[2] は、染色体に作用する操作を「転座 (translocation)」「融合 (fusion)」「分裂 (fission)」「反転 (inversion)」の 4 種類に制限し、遺伝的距離 d_{HP} を定義した。しかしながら、この d_{HP} および提案されたアルゴリズムには、計算上の制約や複雑性に関する課題が存在した。

Bergeron ら (2009)[1] は、これらの問題を解決すべく、上記 4 種類の操作を一般化した「Double-Cut-And-Join (以下、DCJ)」という単一の操作を導入し、新たなアルゴリズムを提案した。DCJ 操作に基づく距離を d_{DCJ} と呼ぶ。

Bergeron らの手法は、染色体内の 8 つの特定パターンを検出することで線形時間での操作手順出力を可能にしているが、その実装は極めて高度であり、直感的な理解が困難な側面がある。本論文では、DCJ 理論における Bergeron らと同様の前提条件の下、C 言語を用いたより簡潔な線形時間アルゴリズムの実装について述べる。本稿の前半では d_{DCJ} の算出に特化したアルゴリズムを、後半では操作手順まで出力する、より汎用性の高いシンプルな実装を紹介する。

実装においては、一部の着想を Bergeron ら (2009) の研究に依拠しつつも、主要なデータ構造および処理ロジックの大部分は独自に構築したものである。

第 1 章　導入

1.1　背景

計算分子生物学において遺伝的距離をモデル化する手法には多くの変遷があるが、その基本的な枠組みは共通している。最終的な目標は、二つのゲノム間の最小変異回数を算出し、それらを比較・評価することにある。

1.2　ゲノムおよび染色体の表現方法

比較対象となる二つのゲノムを、それぞれ *source* および *target* と呼ぶ。各ゲノムは染色体（chromosome）の集合であり、各染色体は遺伝子（gene）の配列として定義される。

各遺伝子は整数値で識別する。最も単純な方法として、n 個の相異なる遺伝子に対し 1 から n までの連番を割り当てる。遺伝子には向き（指向性）が存在するため、符号（正負）を用いてその向きを表現する。

向きの基準は *target* ゲノムに置く。 *target* に含まれる遺伝子の向きを正（+）とし、*source* において *target* と逆向きである場合は負（−)、同一である場合は正（+）とする。これにより、遺伝子数 n のゲノムにおいて、*target* が単一の線形染色体である場合、その表現は常に $\langle 1\ 2\ \ldots\ n \rangle$ と一意に定まる。

線形染色体と環状染色体を区別するため、本論文では以下の表記を用いる。

- **線形染色体**: $\langle \ldots \rangle$ で囲む。例: $\langle 5\ 6\ -1 \rangle$
- **環状染色体**: $(\ldots)$ で囲む。例: $(2\ 7\ -3)$

例えば、以下の遺伝子構成を持つゲノムの数値表現を考える。

$$S : \langle 5\ 6\ -1 \rangle\ (2\ 7\ -3)\ \langle -4 \rangle$$
$$T : \langle 1\ 2\ 3\ 4 \rangle\ (5\ 6\ 7)$$

ここで、環状染色体 (5 6 7) は、要素を巡回させた (6 7 5) や (7 5 6) と同一の染色体を表すものとする（*Remark*)。

1.3 端点 (Extremity) と隣接 (Adjacency)

上述の遺伝子配列による表現は直感的であるが、アルゴリズムの実装においては、遺伝子間の「接続関係」を直接扱う方が効率的である。遺伝子 g の接続箇所を区別するため、各遺伝子に対し二つの **端点 (Extremity)** を定義する。

遺伝子 g に対し、

- **右側の端点**: g （遺伝子の符号と同じ番号）
- **左側の端点**: $-g$ （遺伝子の符号と逆の番号）

とする。例えば、遺伝子 6 の端点は $\{6, -6\}$ であり、遺伝子 -1 の端点は $\{-1, 1\}$ となる。

二つの遺伝子の接続は、これら端点のペアとして表現され、これを **隣接 (Adjacency)** と呼ぶ。配列上で 5 と 6 が隣接している場合、それは端点 5（遺伝子 5 の右側）と -6（遺伝子 6 の左側）の結合 $\{5, -6\}$ を意味する。

また、線形染色体の端（末端）については、特殊なシンボル o を用いて **テロメア (Telomere)** との接続 $\{o, x\}$ として扱う。この定義により、ゲノムは単なる隣接関係の集合として記述でき、線形・環状の区別や染色体数を意識せずに統合的な操作が可能となる。

先述の例を隣接集合として表現すると以下のようになる。

- $S = \{\{o, -5\}, \{5, -6\}, \{6, 1\}, \{-1, o\}, \{2, -7\}, \{7, 3\}, \{-3, -2\}, \{-2, 2\}, \{o, 4\}, \{-4, o\}\}$
- $T = \{\{o, -1\}, \{1, -2\}, \{2, -3\}, \{3, -4\}, \{4, o\}, \{5, -6\}, \{6, -7\}, \{7, -5\}\}$

※ 環状染色体 $(2\ 7\ -3)$ の場合、末尾の -3 と先頭の 2 も接続されるため、$\{-(-3), -2\} = \{3, -2\}$ という隣接が含まれる。

第 2 章　必要な定理

2.1　Bergeron2006 からの定理

2.2　DCJ 理論の数学的基礎

Bergeron ら (2006) は、DCJ 操作を用いたゲノム距離の計算において、以下の重要な定義および定理を示している。

定義 2.1（隣接グラフ $AG(A,B)$）　2 つのゲノム A, B において、それぞれの隣接関係およびテロメアを頂点集合とし、共通の遺伝子端を持つ頂点間にエッジを引いたグラフを隣接グラフと呼ぶ [3]。このグラフは、サイクルとパスの集合となる。

補題 2.2（ゲノムの一致条件）　N 個の遺伝子集合からなるゲノム A, B が $A = B$ であるための必要十分条件は、次式が成り立つことである：

$$N = C + \frac{I}{2} \tag{2.1}$$

ここで、C は隣接グラフ $AG(A,B)$ におけるサイクルの数、I は奇数パス（頂点数が奇数のパス）の数である [3]。

定理 2.3（DCJ 距離公式と計算量）　2 つのゲノム A, B 間の最短の DCJ 操作回数（DCJ 距離）$d_{DCJ}(A,B)$ は、次式で与えられる：

$$d_{DCJ}(A,B) = N - \left(C + \frac{I}{2}\right) \tag{2.2}$$

さらに、この距離を達成する最適な操作手順は、線形時間 $O(N)$ で計算可能である [3]。

2.3　(2.2) 式に関する考察

(2.2) 式は、d_{DCJ} を計算する上で、隣接グラフの初期状態さえあれば、それを変化させることなく、ただ C, I を数えられればよいことがわかる。このことから、私はアル

ゴリズム 1 を提案した。

第 3 章　アルゴリズム 1

このアルゴリズムは、隣接グラフを構築した上で、その中に含まれているサイクル、および奇数長のパスを線形時間で探索するアルゴリズムである。

3.1　必要なデータ構造

Extremity

Adjacency

map

Algorithm 1: 隣接グラフを用いた DCJ 距離の計算

入力: ゲノム A, ゲノム B

出力: DCJ 距離 d_{DCJ}

```
1  N ← 遺伝子数;
2  AG(A, B) ← A と B の隣接集合から隣接グラフを構築;
3  C ← 0 ;                                          // サイクルの数
4  I ← 0 ;                                          // 奇数パスの数
5  foreach 未訪問の隣接ノード v ∈ AG(A, B) do
6      v から始まるパスまたはサイクルをトラバースする;
7      L ← トラバースしたエッジの総数;
8      if サイクルを発見 then
9          C ← C + 1;
10     else if パスを発見 then
11         if L が奇数 then
12             I ← I + 1;
13 d_DCJ = N − (C + I/2);
14 return d_DCJ;
```

3.2　正当性

終了性については、サイクルでの無限ループにさえ気を遣えば、ただの 2N 回の探索なので、明らか。

正当性については、明らか。

第 4 章　アルゴリズム 2

[3] を参考にした。

4.1　必要なデータ構造

Extremity

Algorithm 2: DCJ 操作によるゲノムソート手順の生成 (Implementation of `proceed`)

入力: 現在のゲノム A の端点集合 $exts$, 目標状態 $goals$

出力: 実行された DCJ 操作のログ

$n \leftarrow$ 遺伝子数;

$c \leftarrow 0$; // DCJ 操作のカウンター

// 第 1 段階：目標が通常の端点（非 null）である接続を優先的に処理

for $i = 0$ **to** $2n - 1$ **do**

  $src \leftarrow exts[i]$;

  $tar \leftarrow goals[i]$ に対応する A 上の目標端点;

  if tar **が有効な端点（*nullish* でない）** ***and*** src **の現在の接続相手** $\neq tar$ **then**

    src の現在のペアと tar の現在のペアを特定し、DCJ 操作を実行;

    DoubleCutJoin(A, src, tar);

    $c \leftarrow c + 1$;

    ; // この操作により src と tar が隣接関係になる

// 第 2 段階：目標がテロメア（nullish）である接続を処理

for $i = 0$ **to** $2n - 1$ **do**

  $src \leftarrow exts[i]$;

  $tar \leftarrow goals[i]$ に対応する A 上の目標端点;

  if tar **がテロメア（*nullish* である）** ***and*** src **の現在の接続相手** $\neq tar$ **then**

    src の現在のペアと tar の現在のペア（テロメア）を特定し、DCJ 操作を実行;

    DoubleCutJoin(A, src, tar);

    $c \leftarrow c + 1$;

return c;

4.2 正当性

参考文献

[1] A. Bergeron, J. Mixtacki, and J. Stoye, “A new linear time algorithm to compute the genomic distance via the double cut and join distance,” *Theoretical Computer Science*, 410 (2009), pp. 5300–5316.

[2] S. Hannenhalli and P. A. Pevzner, “Transforming men into mice (polynomial algorithm for genomic distance problem),” in: *Proceedings of FOCS 1995*, IEEE Press, 1995, pp. 581–592.

[3] A. Bergeron, J. Mixtacki, and J. Stoye, “A unifying view of genome rearrangements,” in: *Proceedings of WABI 2006*, LNBI, vol. 4175, Springer Verlag, 2006, pp. 163–173.